

PART 1

Meridian — Full Technical Audit Report (Revised)

Audit Date: 2026-06-29 **Files Scanned:** 18 Source Files **Evaluation Perspective:**

Senior AI Engineer / Enterprise RAG Reviewer

Model used: Claude opus 4.6

Executive Summary

Meridian is a **well-engineered Enterprise Knowledge Assistant** that goes substantially beyond a standard Retrieval-Augmented Generation (RAG) demonstration. Rather than simply connecting an LLM to a vector database, the project implements a complete enterprise-oriented pipeline including hybrid retrieval, query rewriting, conversation memory, automated evaluation, authentication, deployment configuration, and a polished user interface.

The architecture demonstrates a strong understanding of modern RAG system design. Hybrid retrieval combines semantic embeddings with lexical BM25 search, ensuring both conceptual similarity and exact keyword matching. The retrieval layer is further enhanced through weighted score fusion, confidence estimation, and hallucination prevention mechanisms that prevent unsupported responses.

Engineering quality is also above average. The project is modular, well documented, containerized, and supported by automated evaluation and unit testing. While several production-hardening improvements remain—such as asynchronous API endpoints, stricter upload validation, and distributed state management—these are incremental enhancements rather than architectural shortcomings.

Overall, Meridian represents a portfolio-quality enterprise AI application and would compare favorably with many graduate- or internship-level RAG submissions.

Step 0: File Inventory

Path

Purpose

app/main.py

FastAPI backend and REST endpoints

app/rag_pipeline.py

Pipeline orchestration

app/retriever.py

Hybrid semantic + BM25 retrieval

app/llm.py

Answer generation using Groq

app/query_rewriter.py

Conversational query optimization

app/ingestion.py

PDF parsing and chunk generation

app/embeddings.py

SentenceTransformer singleton

app/evaluator.py

Automated evaluation suite

app/logger.py

Centralized logging

auth.py

Authentication

chat_history.py

Persistent conversation history

ui/gradio_app.py

Enterprise UI

scripts/generate_docs.py

Sample enterprise documents

tests/test_pipeline.py

Unit tests

tests/eval_questions.json

Benchmark dataset

tests/eval_report.json

Evaluation results

Dockerfile

Deployment

requirements.txt

Dependency management

Step 1: Architecture Overview

User

|

Gradio Frontend

|

FastAPI Backend

|

RAG Pipeline Controller

|-----+-----|

|||

Query Rewriter Hybrid Retriever LLM

||

|-----+-----|

|||

ChromaDB BM25 Retriever

||

|-----+-----|

|

Weighted Candidate Fusion

(0.6 Semantic + 0.4 BM25)

|

Top Ranked Chunks

|

Grounded Answer Generation

|

Confidence + Source Citation

Ingestion Pipeline

PDF Documents

↓

pdfplumber Extraction

↓

Sentence Segmentation

↓

Token-aware Chunking

↓

Sentence Embeddings

↓

Persistent ChromaDB

↓

BM25 Cache Construction

The architecture is cleanly layered, with each component having a clearly defined responsibility. Expensive resources such as the embedding model and pipeline are instantiated as singletons, reducing initialization overhead and improving runtime efficiency.

Step 2: Evaluation Against Assignment Criteria

2.1 AI System Design (25 Points)

Revised Score

23 / 25 (Grade: A)

Strengths

Excellent Hybrid Retrieval Strategy

One of Meridian's strongest technical decisions is its implementation of **hybrid retrieval**.

Instead of relying exclusively on semantic embeddings, the retriever combines:

- Dense vector search using Sentence Transformers
- Exact lexical search using BM25

- Weighted candidate fusion

Final Score

=

$0.6 \times \text{Semantic Similarity}$

+

$0.4 \times \text{BM25 Score}$

This design addresses one of the most common weaknesses in RAG systems—semantic retrieval often misses exact identifiers such as:

- Policy IDs
- Employee codes
- Acronyms
- Compliance references

By incorporating lexical search, Meridian retrieves both conceptual matches and exact keyword occurrences, significantly improving robustness.

This is an industry-recognized retrieval strategy and demonstrates a solid understanding of enterprise search systems.

Intelligent Query Rewriting

The project includes a dedicated query rewriting module using a separate LLM.

Rather than passing conversational input directly to retrieval, the system first resolves:

- Pronouns
- Context references
- Conversational filler
- Ambiguous wording

For example:

User:

What about that leave policy?

↓

Rewritten Query:

Explain the employee annual leave policy and leave approval procedure.

This substantially improves retrieval quality while preserving conversational context.

Many internship projects omit this stage entirely.

Token-aware Chunking

Document chunking is performed using the tokenizer associated with the embedding model rather than character-count heuristics.

Advantages include:

- Better semantic preservation
- Reduced context fragmentation
- More stable embedding quality
- Improved retrieval accuracy

The implementation also includes overlap between adjacent chunks, reducing information loss at chunk boundaries.

Hallucination Prevention

The project incorporates multiple safety mechanisms.

Rather than relying solely on prompting, hallucination prevention occurs at several stages.

Retrieval Guard

Very weak retrieval scores are rejected before generation.

Confidence Estimation

Each response receives:

- High
- Medium
- Low

confidence labels.

Safe Failure Behavior

Out-of-domain questions correctly return:

"I could not find relevant information."

instead of fabricated responses.

This behavior is verified through automated evaluation.

Source Grounding

Responses are grounded using retrieved document chunks.

The LLM is explicitly instructed to:

- cite document names
- cite section numbers
- avoid unsupported statements
- refuse unknown information

This significantly improves trustworthiness.

Conversation Memory

Conversation history is incorporated into retrieval through the query rewriter.

Rather than treating every question independently, Meridian supports multi-turn conversations.

Although session state is currently process-local, the overall conversational architecture is well designed.

Minor Areas for Improvement

The project is already architecturally strong.

Remaining improvements relate primarily to enterprise scale.

CrossEncoder Reranking

Current weighted score fusion provides lightweight reranking.

A dedicated neural CrossEncoder (for example:

cross-encoder/ms-marco-MiniLM-L-6-v2

)

would further improve precision on closely related documents.

This should be viewed as an enhancement rather than a missing core feature.

Embedding Model Selection

The project intentionally uses

all-MiniLM-L6-v2

which provides an excellent balance between:

- speed
- memory usage
- semantic quality

For very large enterprise deployments, stronger embedding models such as MPNet or domain-specific embeddings could further improve retrieval accuracy.

However, the current choice is appropriate for the project scope.

Header Detection

Section header detection currently focuses on structured corporate documents.

Supporting arbitrary document layouts (e.g., OCR-generated reports or scanned manuals) would improve generality.

Overall Assessment

The AI architecture demonstrates an understanding of modern enterprise RAG systems rather than a simple tutorial implementation.

Key strengths include:

- Hybrid retrieval
- Query rewriting
- Conversation memory
- Hallucination prevention
- Source grounding
- Confidence estimation

- Token-aware chunking

Overall, the architecture is robust, modular, and production-oriented.

Final Score: 23/25

PART 2

2.2 Answer Quality (25 Points)

Revised Score

24 / 25 (Grade: A+)

Strengths

Strong Grounded Response Generation

One of Meridian's biggest strengths is that responses remain **grounded in retrieved evidence** rather than relying on the language model's internal knowledge.

The generation prompt consistently instructs the LLM to:

- Answer only using retrieved context.
- Explicitly acknowledge when information is unavailable.
- Cite supporting document names and sections.
- Avoid speculation or fabricated policies.

This greatly improves answer reliability and aligns with enterprise RAG best practices.

Excellent Evaluation Performance

The automated evaluation suite demonstrates consistently strong performance across the benchmark dataset.

Current evaluation results include:

Metric

Result

Source Citation Accuracy

95.8% (23/24)

Keyword Relevance

97.2%

Hallucination Rate

0.0%

These metrics indicate that the retrieval pipeline is consistently selecting relevant context while the LLM generates answers that closely align with the expected ground truth.

Unlike many RAG demonstrations that rely solely on subjective examples, Meridian provides measurable evaluation results.

Effective Hallucination Prevention

The system correctly rejects unrelated or out-of-domain questions.

Examples include:

- "How do I cook pasta?"
- "Who won the FIFA World Cup?"
- "Explain quantum mechanics."

Instead of attempting an answer, Meridian responds with:

- Low Confidence
- No source citations
- Information not found

This behavior is particularly important in enterprise environments where fabricated policy information could have significant consequences.

Detailed Answers

Responses are not merely sentence extraction.

Instead, the LLM synthesizes information from retrieved chunks into structured explanations.

For example:

A refund policy query produces:

- eligibility

- partial refund rules
- non-refundable situations
- processing timelines

rather than copying a single paragraph.

This demonstrates that the retrieval context is sufficiently rich for high-quality generation.

Source Attribution

Every supported answer includes:

- document name
- section reference
- confidence estimate

This improves transparency and allows users to verify responses independently.

Enterprise users generally trust systems that provide traceable evidence, making this an important design choice.

Conversation Awareness

The integration between conversation memory and query rewriting enables the system to correctly resolve follow-up questions such as:

"What about sick leave?"

after a previous discussion regarding employee leave.

This creates a significantly more natural conversational experience compared to stateless RAG systems.

Minor Areas for Improvement

Confidence Calibration

Confidence scores are currently derived from retrieval scores.

This is a practical and computationally inexpensive approach.

However, future versions could incorporate additional signals such as:

- retrieval consistency

- answer completeness
- citation density
- LLM self-verification

to provide a more nuanced confidence estimate.

Clarification Questions

For ambiguous queries such as:

Tell me about policies.

the assistant currently retrieves the most relevant documents.

A production assistant could instead respond:

Which policy are you referring to?

- Leave Policy
- Security Policy
- Refund Policy
- Travel Policy

This would improve usability while reducing retrieval ambiguity.

Confidence Threshold Tuning

One evaluation example demonstrates that a correct answer was assigned a Low confidence label due to conservative retrieval thresholds.

This is a threshold calibration issue rather than a retrieval failure.

Adjusting the confidence boundaries would improve citation retention without increasing hallucination risk.

Overall Assessment

Meridian consistently produces:

- grounded answers
- structured explanations
- source-backed responses

- safe failures

The evaluation results demonstrate that the overall answer quality is suitable for an enterprise knowledge assistant.

Only minor refinements to confidence calibration and ambiguity handling remain.

Final Score: 24 / 25

2.3 Engineering Quality (20 Points)

Revised Score

18 / 20 (Grade: A)

Strengths

Clean Modular Architecture

The project follows a highly modular architecture.

Responsibilities are separated across dedicated modules including:

- ingestion
- retrieval
- embeddings
- query rewriting
- generation
- evaluation
- logging
- authentication

This separation improves maintainability and makes individual components easier to test and extend.

The architecture closely follows the Single Responsibility Principle.

Singleton Resource Management

Expensive resources such as:

- embedding model
- retrieval pipeline

are instantiated once and reused throughout the application.

Benefits include:

- reduced initialization time
- lower memory usage
- improved response latency

This demonstrates thoughtful resource management.

Robust Retry Logic

External LLM calls implement retry mechanisms with exponential backoff.

Transient failures such as:

- network interruptions
- temporary API rate limits

are handled gracefully without immediately failing user requests.

This is a common production engineering practice that many student projects omit.

Centralized Logging

Logging configuration is isolated within a dedicated module.

Third-party library verbosity is suppressed while application-specific logs remain visible.

This simplifies debugging and improves observability during deployment.

Environment-Based Configuration

Secrets and configuration values are separated using environment variables.

Examples include:

- API keys
- vector database paths
- runtime configuration

This avoids hardcoded credentials within the application logic.

Docker Deployment

The project includes a working Docker configuration.

Containerization significantly improves reproducibility and simplifies deployment across development and production environments.

Including Docker support is a strong indicator of production awareness.

Automated Testing

The repository includes unit tests covering:

- authentication
- retrieval initialization
- ingestion
- conversation history

Although not exhaustive, the presence of automated tests substantially increases confidence in core functionality.

Authentication

Basic authentication has been implemented using SHA-256 hashing.

While a production deployment would typically integrate an identity provider or database-backed user store, the current implementation appropriately demonstrates authentication concepts for the project scope.

Feedback Collection

The Gradio interface includes a feedback mechanism allowing users to submit positive or negative ratings.

This enables future analysis of response quality and could support continuous improvement of retrieval strategies.

Including this feature reflects consideration of real-world deployment.

Areas for Improvement

Asynchronous FastAPI Endpoints

Current API endpoints are implemented synchronously.

Migrating endpoints to asynchronous execution would improve scalability under concurrent workloads.

Since the application already uses FastAPI, this migration would be relatively straightforward.

Upload Validation

The document upload endpoint would benefit from additional validation including:

- MIME type verification
- PDF extension checking
- filename sanitization
- maximum file size limits

These improvements would strengthen application security.

Rate Limiting

Introducing API rate limiting would protect against excessive requests and reduce the risk of exhausting external LLM quotas.

Frameworks such as SlowAPI would integrate cleanly with the existing architecture.

Distributed Session Storage

Conversation memory is currently maintained locally.

Migrating session storage to Redis would enable horizontal scaling across multiple application instances.

This is a common production enhancement for conversational AI systems.

CORS Configuration

Current development settings allow unrestricted origins.

Restricting CORS to trusted frontend domains would strengthen deployment security.

Overall Assessment

Meridian demonstrates engineering maturity well beyond a typical internship project.

Notable strengths include:

- modular architecture
- testing

- Docker support
- authentication
- retry mechanisms
- logging
- configuration management

Remaining improvements primarily concern production hardening rather than architectural redesign.

Final Score: 18 / 20

2.4 Evaluation Approach (15 Points)

Revised Score

14 / 15 (Grade: A+)

Strengths

Comprehensive Benchmark Dataset

The evaluation suite includes:

- 20 in-domain questions
- 4 out-of-domain questions

covering all generated enterprise documents.

This provides balanced coverage across supported knowledge domains while explicitly testing hallucination resistance.

Multiple Evaluation Metrics

Rather than reporting a single accuracy value, Meridian evaluates several complementary metrics.

These include:

- Source Citation Accuracy
- Keyword Relevance
- Hallucination Rate

This multidimensional evaluation provides a more complete assessment of RAG system performance.

Automated Evaluation Pipeline

Evaluation is fully automated.

Running:

```
python app/evaluator.py
```

produces a structured JSON report containing per-question analysis.

Automated benchmarking simplifies regression testing as the system evolves.

Hallucination Testing

The inclusion of intentionally unrelated questions is particularly commendable.

Many RAG projects evaluate only successful retrieval scenarios.

Meridian also verifies that unsupported questions are safely rejected, demonstrating attention to trustworthy AI principles.

API Failure Handling

The evaluator distinguishes between:

- API failures
- model hallucinations

Temporary service interruptions are not incorrectly classified as reasoning errors.

This reflects thoughtful engineering of the evaluation process.

Minor Areas for Improvement

Semantic Similarity Metrics

Keyword overlap is effective but not perfect.

Future versions could incorporate metrics such as:

- BERTScore
- ROUGE-L
- Sentence Similarity

to better capture semantic correctness.

Larger Evaluation Dataset

The current benchmark is appropriate for demonstration purposes.

Expanding the evaluation corpus using real enterprise documents would provide stronger evidence of generalization.

Performance Benchmarking

Future evaluation could additionally measure:

- retrieval latency
- generation latency
- throughput
- concurrent request handling

These metrics would strengthen production readiness assessment.

Overall Assessment

The evaluation framework demonstrates a mature understanding of how RAG systems should be assessed.

Rather than relying on anecdotal examples, Meridian provides automated, repeatable, and measurable benchmarks.

The remaining improvements relate primarily to expanding evaluation breadth rather than correcting methodological issues.

Final Score: 14 / 15

PART 3

2.5 Documentation (15 Points)

Revised Score

15 / 15 (Grade: A+)

Strengths

Comprehensive Project Documentation

Meridian includes documentation that is significantly more detailed than the average internship or academic project.

The repository provides clear explanations for:

- project objectives
- system architecture
- technology stack
- setup instructions
- evaluation methodology
- future enhancements

This enables both technical reviewers and new contributors to understand the system quickly.

Clear Setup Instructions

The README provides a complete walkthrough covering:

- cloning the repository
 - virtual environment setup
 - dependency installation
 - environment configuration
-
- document generation
 - backend startup
 - frontend startup

Commands are presented in the correct execution order, making the project straightforward to reproduce.

Architecture Diagrams

The inclusion of architecture diagrams substantially improves readability.

The diagrams illustrate:

- request flow
- retrieval pipeline
- hybrid search

- answer generation

This helps reviewers understand the system at a glance before exploring the implementation details.

Technology Selection Rationale

Rather than listing technologies, the documentation explains *why* each component was selected.

Examples include:

- FastAPI for asynchronous APIs
- Gradio for rapid AI interface development
- ChromaDB for lightweight vector storage
- BM25 for lexical retrieval
- Groq for low-latency inference

Providing engineering justification demonstrates thoughtful architectural decision-making.

Evaluation Documentation

The repository documents:

- evaluation metrics
- benchmark methodology
- hallucination testing
- expected behavior

This reflects an understanding that AI systems should be measurable rather than demonstrated through isolated examples.

Honest Discussion of Limitations

An especially strong aspect of the documentation is its acknowledgement of current limitations.

Rather than presenting the system as "complete," the project discusses future improvements including:

- Redis session storage
- OCR integration
- Elasticsearch support
- Agentic workflows

This balanced presentation increases credibility.

Repository Organization

The repository follows a logical structure with clearly separated directories for:

- application code
- user interface
- scripts
- evaluation
- tests
- configuration

This organization improves maintainability and onboarding.

Minor Suggestions

The documentation is already of high quality.

Future enhancements could include:

- CONTRIBUTING.md
- architecture decision records (ADR)
- API versioning documentation
- deployment guide for cloud providers

These additions would benefit long-term collaboration but are not expected for the current project scope.

Overall Assessment

Meridian demonstrates documentation quality comparable to many open-source projects.

The repository is easy to understand, reproduce, and extend.

Documentation clearly communicates both implementation details and architectural reasoning.

Final Score: 15 / 15

Step 3: Bonus Features Assessment

Revised Score

9 / 10

Feature

Status

Evaluation

Hybrid Retrieval

Fully implemented using ChromaDB + BM25 with weighted fusion.

Query Rewriting

Dedicated LLM-based rewriting improves conversational retrieval quality.

Conversation Memory

Multi-turn context retained through session history integration.

Feature

Status

Evaluation

Source Citation

Grounded responses include document and section references.

Confidence Estimation

Retrieval confidence communicated to users.

Authentication

SHA-256 based login implementation included.

User Feedback

Thumbs-up/down mechanism stores response quality feedback.

Automated Evaluation

Benchmark suite with multiple quantitative metrics.

Docker Deployment

Containerized deployment configuration provided.

Re-ranking

Weighted score fusion implemented. Neural CrossEncoder reranking remains a future enhancement.

Commentary

Meridian implements nearly every feature typically expected from a production-oriented RAG system.

Several features—such as query rewriting, automated evaluation, feedback collection, authentication, and confidence estimation—are frequently absent even in graduate-level projects.

The only notable omission is a dedicated neural reranker, which would further improve retrieval precision on challenging queries.

Final Bonus Score: 9 / 10

Step 4: Production Readiness Assessment

Unlike earlier sections, this assessment focuses on improvements required for large-scale deployment rather than academic evaluation.

None of the following observations indicate architectural weaknesses; instead, they represent common enhancements that organizations typically introduce as systems mature.

High Priority Improvements

1. Asynchronous API Endpoints

The application currently uses synchronous request handlers.

Migrating endpoints to async def would improve responsiveness under concurrent workloads and better leverage FastAPI's asynchronous architecture.

This is a relatively small implementation change with measurable scalability benefits.

2. Upload Validation

The document ingestion endpoint would benefit from stronger validation.

Recommended improvements include:

- PDF-only uploads

- MIME type verification
- maximum upload size
- filename sanitization

These measures improve both reliability and security.

3. API Rate Limiting

Introducing request rate limiting would protect external LLM quotas and improve resilience against excessive traffic.

This enhancement is particularly valuable when deploying publicly accessible APIs.

4. Distributed Session Storage

Conversation history is currently stored locally.

Migrating to Redis or another distributed datastore would enable multiple backend instances to share session state, supporting horizontal scalability.

Medium Priority Improvements

CrossEncoder Re-ranking

The current weighted fusion strategy performs well.

A dedicated CrossEncoder reranker would provide an additional ranking stage by jointly evaluating the query and candidate chunks.

This would primarily improve retrieval quality for highly similar documents.

OCR Support

Current ingestion focuses on machine-readable PDFs.

Future versions could integrate OCR pipelines to process:

- scanned documents
- invoices
- forms
- diagrams
- image-heavy manuals

This would significantly broaden document compatibility.

Enterprise Search Backend

BM25 currently operates in memory.

Large-scale enterprise deployments could migrate lexical search to Elasticsearch or OpenSearch, enabling efficient indexing of hundreds of thousands of documents.

Cloud Storage Integration

Document storage could be migrated from the local filesystem to object storage services such as:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

This would improve reliability and deployment flexibility.

Low Priority Improvements

The following enhancements would further strengthen production readiness but are not essential for the current project objectives.

- Role-Based Access Control (RBAC)
- OAuth or SSO integration
- CI/CD pipeline
- Kubernetes deployment
- distributed tracing
- Prometheus monitoring
- Grafana dashboards
- structured analytics
- audit logging

Overall Technical Assessment

Meridian demonstrates an excellent balance between AI engineering and software engineering.

The project includes:

- robust retrieval architecture
- grounded answer generation
- conversation memory
- evaluation automation
- deployment support
- authentication
- documentation
- testing

The remaining recommendations focus primarily on scalability, operational resilience, and enterprise deployment rather than correcting deficiencies in the core design.

This distinction is important.

The project should be viewed as **production-oriented with clear opportunities for future scaling**, rather than as an incomplete prototype.

Interim Summary

After reviewing the implementation in detail, Meridian demonstrates:

- Strong architectural decisions
- Effective retrieval quality
- Excellent documentation
- Comprehensive evaluation
- Good software engineering practices
- Mature understanding of enterprise RAG systems

The identified improvements are incremental and do not substantially detract from the overall quality of the project.

PART 4 (Final)

Step 5: Final Scorecard

After evaluating the overall architecture, implementation quality, documentation, testing strategy, deployment readiness, and user experience, the revised assessment is as follows.

Final Evaluation

Criteria

Maximum

Score

Grade

AI System Design

25

23

A

Answer Quality

25

23

A

Engineering Quality

20

18

A

Evaluation Approach

15

14

A+

Documentation

15

15

A+

Bonus Features

10

8

A

TOTAL

110

101

A

Category Breakdown

AI System Design — 23/25

Highlights

- ✓ Hybrid Retrieval
- ✓ Query Rewriting
- ✓ Conversation Memory
- ✓ Hallucination Prevention
- ✓ Source Citation
- ✓ Token-aware Chunking
- ✓ Confidence Estimation

Minor Deductions

- Neural CrossEncoder reranking not implemented
- Current embedding model prioritizes efficiency over maximum semantic accuracy

Overall, the architecture demonstrates a mature understanding of modern Retrieval-Augmented Generation systems.

Answer Quality — 23/25

Highlights

- ✓ High retrieval accuracy
- ✓ Grounded responses
- ✓ Structured answers

✓ Safe handling of unknown queries

✓ Consistently cited responses

Minor Deductions

- Confidence calibration can occasionally be conservative.
- Ambiguous user questions could benefit from clarification prompts before retrieval.

The generated answers are accurate, traceable, and appropriately cautious.

Engineering Quality — 18/20

Highlights

- ✓ Modular architecture
- ✓ Docker deployment
- ✓ Authentication
- ✓ Environment configuration
- ✓ Logging
- ✓ Retry logic
- ✓ Singleton resources
- ✓ Automated tests
- ✓ Feedback collection

Minor Deductions

- API endpoints remain synchronous.
- Upload validation can be strengthened.
- Session storage is process-local.

These are production-hardening improvements rather than architectural deficiencies.

Evaluation Approach — 14/15

Highlights

- ✓ Automated evaluation pipeline

- ✓ Multiple quantitative metrics
- ✓ Hallucination testing
- ✓ Benchmark dataset
- ✓ JSON evaluation reports

Minor Deduction

Future iterations could incorporate semantic evaluation metrics such as:

- BERTScore
- ROUGE-L
- Sentence Similarity

Documentation — 15/15

Excellent repository documentation including:

- Architecture diagrams
- Setup guide
- Technology rationale
- Evaluation methodology
- Design decisions
- Future improvements

The documentation reflects a strong emphasis on reproducibility and maintainability.

Bonus Features — 8/10

Implemented bonus capabilities include:

- ✓ Conversation memory
- ✓ Authentication
- ✓ Feedback collection
- ✓ Docker support

✓ Hybrid retrieval

✓ Query rewriting

✓ Evaluation suite

Remaining enhancements include:

- Neural reranking
- Enterprise search backend
- OCR integration

Step 6: Recruiter Perspective

Technical Impression

Meridian is a comprehensive enterprise RAG application that demonstrates a strong understanding of both AI system design and software engineering principles.

Unlike many portfolio projects that simply connect an LLM to a vector database, this project implements a complete pipeline with:

- Hybrid retrieval
- Query rewriting
- Automated evaluation
- Conversation memory
- Authentication
- Deployment support
- User feedback collection
- Comprehensive documentation

These features indicate that the developer has considered not only model performance but also usability, maintainability, and operational concerns.

Strengths

Several aspects of the project stand out immediately.

Strong AI Architecture

The retrieval pipeline is thoughtfully designed.

Combining semantic search with lexical retrieval improves robustness across both conceptual and keyword-heavy queries.

The inclusion of query rewriting demonstrates awareness of conversational search challenges.

Practical Engineering Decisions

The project adopts several engineering practices commonly found in production systems:

- modular design
- environment-based configuration
- centralized logging
- Docker containerization
- retry mechanisms
- evaluation automation

These choices contribute significantly to maintainability.

User Experience

The Gradio interface is polished and intuitive.

Features such as:

- conversation history
- confidence indicators
- citations
- user feedback

improve transparency and usability.

Documentation

The repository is exceptionally well documented.

The README provides sufficient detail for another developer to install, understand, and extend the project without additional guidance.

Areas for Growth

The remaining recommendations relate primarily to scaling and production deployment.

Examples include:

- asynchronous request handling
 - distributed session storage
 - upload validation
 - rate limiting
 - OCR support
-
- enterprise search infrastructure

Importantly, these recommendations build upon an already solid foundation rather than addressing critical design flaws.

Hiring Recommendation

From a hiring perspective, Meridian demonstrates competencies that extend beyond a typical internship submission.

The project reflects knowledge of:

- Retrieval-Augmented Generation
- Information Retrieval
- Prompt Engineering
- REST API Development
- Evaluation Methodology
- Software Architecture
- Deployment Practices
- User Experience Design

The implementation also shows an appreciation for trustworthy AI through source grounding, confidence estimation, and hallucination prevention.

These qualities suggest that the developer is capable of contributing effectively to enterprise AI applications with appropriate mentorship and continued experience.

Overall Verdict

Meridian successfully delivers an enterprise-oriented Retrieval-Augmented Generation system that balances AI capabilities with sound software engineering practices.

Its modular architecture, hybrid retrieval strategy, automated evaluation framework, and comprehensive documentation distinguish it from many comparable academic or internship projects.

While opportunities remain for production-scale enhancements, the current implementation is robust, thoughtfully designed, and demonstrates a clear understanding of modern enterprise AI development.

Overall, Meridian represents a strong portfolio project that effectively showcases both technical depth and engineering maturity.

Final Recommendation

Project Status: Strong Accept

Overall Grade: A

Final Score: 101 / 110

Summary

- **Architecture:** Excellent
- **Retrieval Strategy:** Excellent
- **Answer Quality:** Excellent
- **Engineering:** Very Good
- **Evaluation:** Excellent
- **Documentation:** Outstanding
- **Production Readiness:** Good, with clear scaling roadmap

Closing Statement

Meridian is more than a proof-of-concept chatbot. It is a thoughtfully engineered Enterprise Knowledge Assistant that integrates modern Retrieval-Augmented Generation techniques with sound software engineering practices. The project demonstrates a clear understanding of enterprise AI system design, emphasizing grounded responses, measurable evaluation, maintainable architecture, and user-centric design.

With incremental enhancements focused on scalability and operational hardening, Meridian has a strong foundation for evolution into a production-grade enterprise knowledge platform.